

Feature Introduction

What is the feature?

- Teaching **recursion** first as a general problem-solving pattern (fully independent of programming), - and only later connecting it to programming.
- The learner moves through six stages: feeling the need for a new way to solve problems, discovering that way, understanding it deeply through real-world scenarios, and finally connecting and applying it to programming.

Why this topic

- Most programming constructs have some intuitive real-world anchor — a list is "a bunch of things," a loop is "doing something repeatedly." Recursion has no such anchor. No one arrives at "solve a problem by delegating a smaller version of it, and waiting on that smaller version's answer" through ordinary life experience. This makes recursion the sharpest possible test of a discovery-driven, need-before-syntax teaching philosophy — if the pattern can be made to feel inevitable rather than arbitrary, it proves something about the *method*, not just this one topic.
- It is also one of the most common sources of persistent confusion in CS education. Research on novice mental models of recursion identifies a common misconception — the **"looping model"** (mistaking recursion for repetition) — as opposed to the correct **"copies model"** (each call as a distinct instance, waiting on a smaller one to resolve). Teaching the pattern with no code and no programming vocabulary at all removes the crutch that lets learners misapply a loop-shaped intuition, forcing the correct mental model to form on its own terms, before it can be contaminated by a resemblance to iteration.

Focus & USP

1. Recursion is taught as a general problem-solving pattern before it is ever named or connected to programming.
2. No code, anywhere — only diagrams, charts, or at most pseudocode, even after the programming connection is made.
3. No programming vocabulary in Stages 1–4 ("loop," "base case," "algorithm," etc.) — plain language only, so the pattern is never presented as inherently computational.

How we intend to teach it

Stage 1 — Feel the need

- *Contains*: a real-world problem where naive, one-at-a-time approaches become tedious or intractable.
- *Focus*: the tedium/impossibility itself, not any solution.
- *Intent*: make the learner genuinely feel that the obvious way of solving this doesn't scale — with no connection to programming at all.

Stage 2 — Discover the approach

- *Contains*: the same problem from Stage 1, solved a new way.

- *Focus*: the felt contrast between "the old way" and "this new way" — recursion remains unnamed throughout.
- *Intent*: let the learner experience the new approach as genuinely easier, building intuition and curiosity rather than handing them a definition.

Stage 3 — Reveal the concept

- *Contains*: the name (recursion), plus further stories, case studies, and activities.
- *Focus*: how the pattern works, what kinds of problems it suits, what kinds of problems it probably shouldn't be used for, and why it always needs a stopping point and a delegating step (base case / recursive case — still unnamed at this stage).
- *Intent*: turn the intuition from Stage 2 into a properly understood concept, with its boundaries and requirements made explicit.

Stage 4 — A satisfying real-world application

- *Contains*: one full, real-world scenario demonstrating the mature pattern end to end.
- *Focus*: every concept introduced in Stage 3, made explicit as it appears in this one worked example.
- *Intent*: leave the learner with a deep, settled understanding of the concept before programming enters the picture at all.

Stage 5 — Connect to programming

- *Contains*: recursion applied to problems the learner can already solve another way — factorial, sum of natural numbers.
- *Focus*: mapping the already-understood pattern onto programming terms.
- *Intent*: introduce recursion as an alternative tool for familiar problems, not a replacement for what the learner already knows.

Stage 6 — Where it's the real deal

- *Contains*: problems where recursion is the only practical way to solve them cleanly — DFS, maze-solving.
- *Focus*: the payoff — cases where the new tool is genuinely necessary, not just an alternative.
- *Intent*: close the arc by showing recursion earning its place, not just being an interesting alternative.

Audience target

Any learner who understands English and is curious to learn — no CS background or prior computational-thinking exposure required.

Final deliverables

1. A detailed document covering the design philosophy and the full learning-stage breakdown.
2. A software prototype implementing Deliverable 1.